

# Computer Memory Hierarchy — Detailed Lecture Notes

JKSSB Computer Awareness

October 8, 2026

## 07.1 Why a memory hierarchy exists

A running computer never keeps all its data and instructions in one place. Instead it uses several stores arranged in a hierarchy. The reason is a three-way trade-off: the memory that is fastest is also the smallest and the most expensive per bit, while the memory that is largest and cheapest is the slowest. No single memory can be fast, large, and cheap at the same time. The hierarchy places the small, fast, costly memories close to the CPU and the large, slow, cheap memories further away.

**Definition (Memory hierarchy).** *The ordered arrangement of storage levels in a computer, from the fastest and smallest (registers) through cache and main memory down to the slowest and largest (secondary and offline storage), chosen to balance speed, cost, and capacity.*

| Term               | Meaning in this lesson                                                      |
|--------------------|-----------------------------------------------------------------------------|
| Register           | The fastest and smallest storage, located inside the CPU.                   |
| Cache              | A small, very fast memory that holds recently and frequently used data.     |
| Main memory (RAM)  | The primary memory that holds active programs and data; it is volatile.     |
| ROM                | Non-volatile read-only primary memory that holds firmware such as the BIOS. |
| Secondary storage  | Non-volatile storage such as an SSD or HDD, used for files.                 |
| Tertiary / offline | Backup and archival storage such as magnetic tape or cloud backup.          |

The rest of this lesson uses these terms in one consistent sense. A **register** is the CPU's own storage. **Cache** is the small fast memory between the CPU and main memory. **Main memory** means RAM: the primary memory that holds what is running. **Secondary storage** means SSD and HDD, which keep files without power. **Tertiary or offline** storage is the slowest level, used for backup and archive.

## 07.2 The ordered hierarchy

The levels, read from fastest to slowest, are registers, cache, main memory (RAM), secondary storage, and then tertiary or offline storage.

| Level                          | Speed           | Size     | Typical role                      |
|--------------------------------|-----------------|----------|-----------------------------------|
| Registers                      | Fastest         | Smallest | Values in use right now           |
| Cache (L1/L2/L3)               | Very fast       | Small    | Recently and frequently used data |
| Main memory (RAM)              | Moderately fast | Larger   | Active programs and data          |
| Secondary (SSD/HDD)            | Slower          | Large    | Permanent file storage            |
| Tertiary/offline (tape, cloud) | Slowest         | Largest  | Backup and archive                |

Reading downward, capacity tends to grow and speed tends to fall. The anchor to remember is at the top: **registers are the fastest** storage in the computer. Below them cache is still very fast but slower than the registers. Main memory (RAM) is larger than cache but slower. Secondary storage (SSD or HDD) is larger again, cheaper per bit, and slower, but it keeps its contents without power. Tertiary or offline storage (magnetic tape, cloud backup) is the slowest and cheapest level and is used for backup and long-term archive.

### 07.3 Registers and cache

Registers are located inside the CPU and are the fastest and smallest storage in the machine. Because they are so small, they hold only the values needed at this instant: instructions, addresses, data, and intermediate results.

Cache is a small, very fast memory that sits between the CPU and main memory. It holds recently and frequently used data and instructions, and it is faster than main memory. Cache does not store new information of its own; it keeps *copies* of data that really live in RAM. It is therefore not a backup, and it loses its contents when power is removed, just as RAM does.

**Definition (Cache).** *A small, very fast memory that holds recently and frequently used data and instructions so the CPU can avoid the slower trip to main memory.*

Cache is usually described in levels. **L1** is the smallest and fastest and sits closest to the CPU core. **L2** is larger and a little slower than L1. **L3** is the largest and slowest of the three, is often shared across cores, and is still faster than main memory. For exam purposes, the key point is the ordering: L1 is fastest and smallest, and L3 is largest and slowest among the three, but all cache is faster than RAM.

**Example 07.3.1.** *While a student edits a document on the office desktop, the same lines and the nearby rows are read again and again. The cache keeps those recently used and nearby values ready, so the CPU does not have to wait for main memory on every access. The cache holds copies of those values; the master copy remains in RAM.*

### 07.4 Main memory (RAM) and ROM

Main memory is the computer's primary memory. The programs and data that are currently in use are held here while the machine runs, and the CPU works directly with main memory. **RAM** (Random Access Memory) is *volatile*: its contents are lost when the power is switched off. RAM is larger than cache but slower.

**ROM** (Read-Only Memory) is the other part of primary memory. It is *non-volatile*: it keeps its contents without power. In normal operation it can be read but not written; changing it needs a special process. ROM holds firmware such as the startup (BIOS) instructions.

**Definition (Volatile memory).** *Memory that loses its contents when the power is removed. RAM is the common example.*

**Definition (Non-volatile memory).** *Memory that retains its contents without power. ROM, SSD, and HDD are common examples.*

**Example 07.4.1.** *An item states, “RAM is non-volatile and will keep the file after shutdown.” This is wrong. RAM is volatile; its contents are lost when the power goes off. It is **ROM** that is non-volatile. The wrong option has swapped the two primary-memory types.*

## 07.5 Primary memory vs secondary storage

**Primary memory** is directly accessible by the CPU and holds the active programs and data. It includes RAM, which is volatile, and ROM, which is non-volatile. **Secondary storage**, such as an SSD or HDD, is non-volatile and keeps files and programs between sessions. It is larger and cheaper per bit but slower than primary memory. The CPU does not access secondary storage directly; data moves through main memory under the control of the operating system.

## 07.6 Secondary and tertiary storage

Secondary storage comes in two common forms. A **solid-state drive (SSD)** uses flash memory and has no moving parts; a **hard disk drive (HDD)** uses rotating magnetic disks. Both are non-volatile and keep data without power. **Tertiary or offline** storage, such as magnetic tape and cloud backup, is the slowest and cheapest level and is used for backup and long-term archive.

## 07.7 Locality of reference

Programs do not access memory at random; they tend to reuse the same data and instructions. This behaviour is called **locality of reference**, and it comes in two forms. **Temporal locality** means the same item is accessed again soon after it was last used, such as a loop counter used on every iteration. **Spatial locality** means items stored at nearby addresses are accessed next, such as the consecutive elements of an array. This reuse is exactly why cache works: by keeping recently used and nearby data ready, cache reduces the average time the CPU waits for data.

**Definition (Locality of reference).** *The tendency of a running program to reuse recently accessed data (temporal locality) and data at nearby addresses (spatial locality).*

## 07.8 Exam retrieval and common confusions

Five distinctions prevent most errors on this topic.

1. RAM is **volatile**; ROM is **non-volatile**. Do not swap them (TRAP-01).
2. Cache is **not** a backup. It holds copies of data that live in RAM.
3. RAM is main memory, **not** storage for files. Files belong in secondary storage.

4. Cache is **faster** than main memory; reversing this is a common wrong answer (PYQ-03).
5. The order of speed is registers, then cache, then RAM, then secondary storage. Registers are fastest.

**Example 07.8.1.** *An item asks which memory is fastest. Trace the hierarchy from the top: registers are fastest and smallest, then cache, then main memory (RAM), then secondary storage. The answer is registers. If the stem instead asks which level holds recently used data, the answer is cache, and if it asks which is volatile primary memory, the answer is RAM.*

## Exercises

1. List the levels of the memory hierarchy from fastest to slowest.
2. Explain why the faster memories of the hierarchy are placed close to the CPU.
3. Distinguish volatile memory from non-volatile memory, giving one example of each.
4. Why is cache described as faster than main memory, and why is it not a backup?
5. Define temporal locality and spatial locality, and explain how they relate to cache.

## Solutions

1. Registers, cache (L1, L2, L3), main memory (RAM), secondary storage (SSD/HDD), then tertiary/offline storage.
2. Faster memory is smaller and more expensive per bit. Placing it close to the CPU keeps the values the CPU needs soonest at the fastest level, so the average access time is low while cost and capacity are balanced.
3. Volatile memory loses its contents when power is removed — RAM is the example. Non-volatile memory retains its contents without power — ROM, or an SSD/HDD, is the example.
4. Cache sits between the CPU and main memory and is a smaller, faster technology, so it is faster than RAM. It only keeps copies of data that already exist in RAM, so if the cache is lost the data is still in RAM; it is therefore not a backup.
5. Temporal locality is the reuse of the same item soon after it was last accessed; spatial locality is the use of data at nearby addresses. Because programs show both patterns, keeping recently used and nearby data in the small fast cache pays off.